ATTS: Agentic Test-Time Scaling via Adaptive Exploration

Peijia Qin
University of California, San Diego
pqin@ucsd.edu

Abstract

Test-time scaling has become a key mechanism for improving large language model reasoning, but existing approaches often rely either on rigid heuristic allocation rules or on explicit fine-grained search over intermediate reasoning steps. We introduce ATTS, a delegated sequential test-time scaling framework in which a lightweight orchestrator adaptively controls a pool of candidate solutions through a single action: `explore`, which launches a fresh, independent solver. The orchestrator decides when to stop exploring and directly synthesizes the final answer from the accumulated evidence. By restricting control to this minimal interface, ATTS shifts compute allocation toward semantic evidence management over candidate hypotheses. We evaluate ATTS on six benchmarks spanning mathematical reasoning, scientific question answering, code generation, and multimodal reasoning, and position it against direct generation, majority voting, self-refinement, budget forcing, and reward-model-based reranking baselines.

1 Introduction

The allocation of inference-time compute has emerged as a central frontier for improving the reasoning capabilities of large language models. Recent work on test-time compute is commonly organized around sequential scaling, parallel scaling, and hybrid search, and the compute-allocation perspective of Snell et al. [15] makes clear that no single strategy is uniformly optimal across prompts of varying difficulty. At the same time, simply allocating more compute is not enough: naive scaling can saturate, and in sampling-based settings can even degrade through poor coverage and reward hacking [5]. The central question is therefore not whether to scale test-time compute, but how to allocate it adaptively and efficiently.

Existing approaches often answer this question with either heuristics or explicit micro-level search. Predictive routing methods such as DiffAdapt use lightweight difficulty estimates to choose an inference regime before generation begins, but these decisions remain open-loop and cannot react to evidence that appears during reasoning [10]. At the other extreme, Tree of Thoughts and process-reward frameworks such as Math-Shepherd perform closed-loop adaptation through explicit tree expansion, pruning, and step-level verification [23, 19]. Recent agentic test-time scaling methods add another variant of this pattern, but many still rely on statistical control rules such as vote entropy, confidence-triggered halting, or parallel mixtures of tool-using agents [7, 4, 8]. An underexplored alternative is a lightweight controller that allocates compute over a semantic pool of candidate hypotheses without explicit search trees or handcrafted stopping thresholds.

We introduce ATTS, a delegated sequential test-time scaling framework that separates *control* from *solution generation*. A lightweight orchestrator manages a growing pool of candidate solutions through a single tool, `explore`, which dispatches a fresh, independent solver on the original problem. When the orchestrator judges the pool to be sufficiently informative, it stops exploring and synthesizes the final answer directly from the accumulated evidence. Echoing Sutton's "Bitter Lesson" [16], our

Preprint. Under review.

claim is not that richer control languages are necessary, but that a deliberately simple action space can be functionally preferable when compute allocation should depend on semantic evidence rather than brittle heuristics.

We evaluate ATTS across six benchmarks spanning mathematical reasoning, scientific question answering, code generation, and multimodal reasoning: HLE-Verified [24], LiveCodeBench [6], BabyVision [3], AIME 2025 and AIME 2026[1], and GPQA-Diamond [13]. Because the controller is benchmark-agnostic, the same policy transfers across text-only and image-conditioned tasks.

2 Related Work

Recent work on test-time compute for large language models is commonly discussed under three broad families: sequential scaling, parallel scaling, and hybrid search. Following the compute-allocation perspective of Snell et al. [15], we use this taxonomy only to position ATTS relative to existing inference-time strategies.

Sequential Scaling. Sequential scaling increases reasoning depth by allocating additional computation along a single evolving trajectory. The defining property is causal dependence across steps: later computation conditions on earlier intermediate outputs, so the extra budget is spent on iterative refinement, longer deliberation, or explicit backtracking rather than on independent samples. Representative examples include self-correction and revision methods such as Self-Refine [11], as well as long-thinking or budget-controlled decoding methods such as s1 with Budget Forcing [12]. Recent benchmark work also sharpens the empirical picture of this family: CorrectBench shows that self-correction can help on hard reasoning tasks, but that sophisticated correction pipelines often deliver limited gains relative to their added cost, and that simple CoT baselines remain surprisingly competitive [17]. In this family, test-time scaling is achieved by making one reasoning path deeper, more reflective, or more persistent before committing to a final answer.

Parallel Scaling. Parallel scaling increases reasoning width by drawing multiple independent candidates and then aggregating or reranking them. The classical instance is self-consistency, which samples diverse reasoning paths and marginalizes over their final answers [21]. More recent work pairs this width expansion with learned reward models or verifier models that score full responses, enabling majority-vote selection rather than majority voting. This line includes outcome-reward reranking in the compute-scaling analysis of Snell et al. [15] and modern open reward models designed explicitly for Majority Voting inference, such as Skywork-Reward-V2 [9]. Recent theoretical work further shows that naive Majority Voting scaling is not uniformly monotonic: its effectiveness depends on policy coverage over good responses, and overly aggressive scaling can degrade performance through reward hacking [5]. Complementarily, Chen et al. [2] argue that this issue is not only an inference-time phenomenon but also a training-time one, showing that standard cross-entropy fine-tuning can become misaligned with pass@N scaling by making models overconfident. In multimodal settings, the same pattern appears in VisualPRM, which scores candidate multimodal reasoning traces for reranking [20]. The common structure is that candidates are generated independently and the additional compute is spent on breadth plus post hoc selection.

Hybrid Search. Hybrid methods combine depth and width by repeatedly expanding multiple partial trajectories and pruning them with search or reward signals. Tree of Thoughts is an early and influential example: it generalizes chain-of-thought into a tree over intermediate "thoughts," allowing lookahead, self-evaluation, and backtracking [23]. Subsequent work strengthens this paradigm with more explicit search and value estimation, for example by integrating Monte Carlo Tree Search with iterative preference learning [22] or by introducing step-level process reward models such as Math-Shepherd to provide dense supervision over intermediate reasoning steps [19]. More recent hybrid systems such as Recursive Self-Aggregation explicitly combine population-level candidate aggregation with iterative refinement, showing that depth and width can be interleaved rather than treated as disjoint scaling axes [18]. These methods occupy the middle ground between pure sequential refinement and pure parallel sampling. We position ATTS against this landscape as a delegated sequential controller rather than an explicit tree-search method.

[1] `https://matharena.ai/competition_tables/aime--aime_2025`
`https://matharena.ai/competition_tables/aime--aime_2026`

Agentic Test-Time Scaling. Recent work has begun to study test-time scaling explicitly for agents. CATTS allocates compute using vote-derived uncertainty statistics such as entropy and top-1/top-2 margin [7]; TUMIX scales through parallel heterogeneous agents with confidence-based early stopping [4]; and General AgentBench provides a broader empirical study of sequential and parallel scaling for open-ended LLM agents [8]. These works establish that agent settings introduce new scaling bottlenecks, but they predominantly rely on statistical routing rules or parallel mixtures. In contrast to these designs, ATTS focuses on reasoning benchmarks where the central object of control is a semantic pool of candidate hypotheses, and where adaptive compute allocation is expressed through a single action—`explore`—rather than explicit search trees or handcrafted uncertainty thresholds.

3 Methodology

3.1 Problem Formulation

Given an input problem x, the goal is to produce a final answer y under a bounded test-time budget. ATTS maintains a state $S_t = (x, \mathcal{C}_t, b_t)$, where $\mathcal{C}_t = \{c_1, \ldots, c_{n_t}\}$ is the current candidate pool and b_t is the remaining exploration budget. Each candidate $c_i = (a_i, r_i, p_i, s_i)$ contains an answer, free-form reasoning, an approach summary, and an optional self-reported confidence score.

3.2 The ATTS Loop

The method proceeds as follows. A lightweight orchestrator repeatedly decides whether to call `explore` or to stop and synthesize. It never solves the problem itself.

Explore. Each `explore` call dispatches a fresh solver on the original problem x. The solver works from scratch—it does not see or continue previous candidates—and returns a structured candidate that is appended to $\mathcal{C}_t$. This fresh-start design encourages diversity: different strategies, error modes, and confidence profiles emerge independently across candidates.

Stopping. The orchestrator can stop at any point before the exploration budget T is exhausted, for example when candidates converge, when one candidate dominates with high confidence, or when further exploration is unlikely to change the decision. If the budget is exhausted without an explicit stop, the method falls back to the highest-confidence candidate in the pool.

Synthesis. When the orchestrator stops, it synthesizes the final answer directly from $\mathcal{C}_t$. Because it has observed the full multi-turn history—each candidate's arrival, its reasoning, and its own deliberation about whether to continue—it has richer context than a separate aggregator that only sees the final pool. The orchestrator compares candidates, reconciles partial evidence, and produces a final answer that reflects the aggregate evidence.

The control logic is benchmark-agnostic: only the explorer prompt and answer schema change across tasks. Appendix A gives the full implementation details.

4 Experiments

We evaluate on six benchmarks: HLE-Verified [24], LiveCodeBench [6], BabyVision [3], AIME 2025 and AIME 2026[2], and GPQA-Diamond [13]. The main text presents results on HLE-Verified, LiveCodeBench, BabyVision, and GPQA-Diamond, which collectively span mathematical reasoning, scientific question answering, code generation, and multimodal reasoning. AIME 2025 and AIME 2026 are deferred to Appendix B because the backbone model already achieves near-ceiling accuracy on these 30-problem benchmarks, limiting the statistical resolution for method comparison. Appendix C collects the full official task definitions, grading rules, source repositories, and leaderboard alignment notes for all benchmarks.

[2] `https://matharena.ai/competition_tables/aime--aime_2025`
`https://matharena.ai/competition_tables/aime--aime_2026`

4.1 Experimental Setup

Across all experiments, the underlying backbone model is held fixed and only the inference-time strategy changes. Pass@1 uses a single model rollout. For benchmarks whose official answer format supports answer-level aggregation, Majority Voting draws N independent candidates from the same base model and answer interface, then selects a final response without any extra training. Self-Refine reuses the same model in a feedback-then-revision loop [11]. Budget Forcing follows the compute-control strategy of Muennighoff et al. [12]: it enforces a reasoning budget by either truncating the model's thinking process or extending it by suppressing the end-of-thinking delimiter and appending "Wait" so that the model continues deliberating before producing a final answer. We additionally include two reward-model baselines: Skywork-Reward-V2, which uses the released Skywork-Reward-V2-Qwen3-8B reward model [9][3], and VisualPRM, which uses the released VisualPRM-8B multimodal process reward model [20][4]. For ATTS, the controller may issue up to a fixed exploration budget of `explore` calls before stopping and synthesizing the final answer directly, as described in Section 3. Across these rows, the comparison isolates differences in test-time inference strategy rather than differences in training data, parameter count, or post-training recipe. In all tables, **bold** marks the best accuracy per group (Zero-Shot Models and Test-Time Scaling Methods separately) and the lowest cost per question among test-time scaling methods.

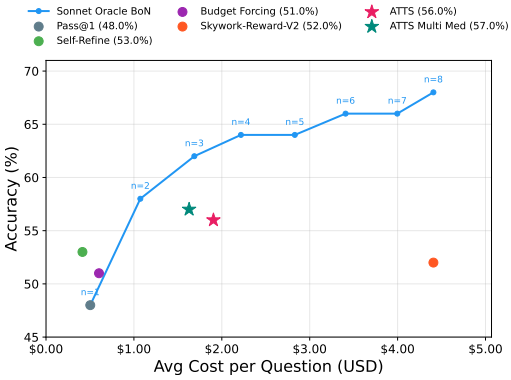
4.2 Main Results

HLE-Verified. We evaluate ATTS on the HLE-Verified Gold setting and report Accuracy. Table 1 also reproduces the official revised-subset references from Zhai et al. [24] for context; the full HLE-Verified subset definitions, official prompt/evaluation protocol, and the revised-versus-gold comparison caveat are summarized in Appendix, HLE-Verified.

Table 1: Baseline results are reported on Revised: HLE-Verified Subset Acc, while the aligned rows for the baselines and ATTS are reported on a 100-sample subset from the HLE-Verified Gold set. Cited from Zhai et al. [24] and the official dataset card. The official revised-subset references are shown here for context and are not directly comparable to Gold-subset evaluations.

	Acc. (%)	$/q
Zero-Shot Models		
Gemini 3 Pro	48.93	–
GPT-5.2-High	**52.48**	–
Claude Opus 4.5	48.16	–
Grok 4.1 Fast Reasoning	43.07	–
Claude Opus 4.6	50.16	–
DeepSeek-V3.2	47.45	–
Qwen3-Max-Thinking	48.48	–
Test-Time Scaling Methods		
Pass@1	48.00	**0.28**
Self-Refine	53.00	0.41
Budget Forcing	51.00	0.60
Skywork-Reward-V2	52.00	2.22
ATTS (Ours)	56.00	1.90
ATTS Multi-model Med (Ours)	**57.00**	1.63

Figure 1: HLE-Verified cost-accuracy tradeoff.



On HLE-Verified, ATTS achieves 56.00% accuracy at $1.90/q, outperforming Self-Refine (53.00%) and Budget Forcing (51.00%). The multi-model variant with medium exploration effort reaches the highest accuracy (57.00%) at comparable cost ($1.63/q). The strong performance on this challenging benchmark suggests that ATTS's adaptive compute allocation is most beneficial when problems

[3] `https://huggingface.co/Skywork/Skywork-Reward-V2-Qwen3-8B` `https://github.com/SkyworkAI/Skywork-Reward-V2`

[4] `https://huggingface.co/OpenGVLab/VisualPRM-8B` `https://internvl.github.io/blog/2025-03-13-VisualPRM/`

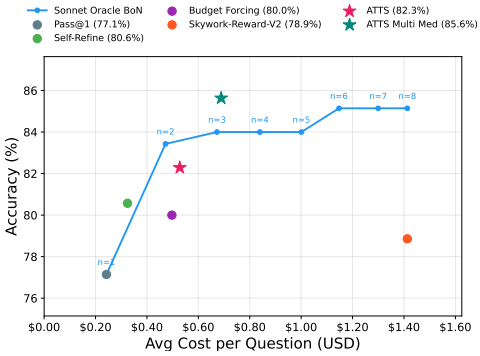
are difficult enough that multiple independent solution attempts yield complementary reasoning fragments.

LiveCodeBench. For LiveCodeBench, we use the official code-generation setup and report overall and difficulty-wise Pass@1. Table 2 follows the public generation leaderboard view; the dataset release, contamination-aware time-window construction, and exact execution-based grading protocol are deferred to Appendix, LiveCodeBench.

Table 2: LiveCodeBench official leaderboard references together with aligned reporting rows for the baselines and ATTS. Cited from Jain et al. [6] and official online records.

	Pass@1	Easy	Med.	Hard	$/q
Zero-Shot Models					
O4-Mini (High)	**80.2**	**99.1**	**89.4**	**63.5**	–
O3 (High)	75.8	**99.1**	84.4	57.1	–
O4-Mini (Med.)	74.2	98.2	86.5	52.7	–
Gemini-2.5-Pro	73.6	**99.1**	87.2	50.2	–
DeepSeek-R1	73.1	98.7	85.2	50.7	–
Test-Time Scaling Methods					
Pass@1	74.29	95.35	86.54	55.00	**0.24**
Self-Refine	80.57	**100.00**	90.38	63.75	0.32
Budget Forcing	80.00	**100.00**	88.46	63.75	0.50
Skywork-Reward-V2	78.86	**100.00**	90.38	60.00	1.41
ATTS (Ours)	81.71	**100.00**	90.38	67.50	0.53
ATTS Multi Med (Ours)	**85.63**	**100.00**	**92.31**	**73.42**	0.69

Figure 2: LiveCodeBench cost-accuracy tradeoff.



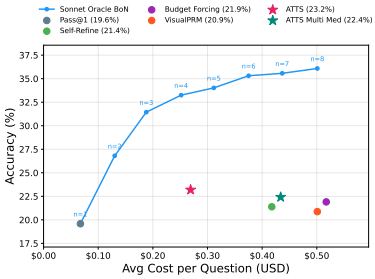
On LiveCodeBench, ATTS achieves 81.71% at $0.53/q, outperforming Self-Refine (80.57%) and Budget Forcing (80.00%), with 100% on Easy, 90.38% on Medium, and the highest single-model Hard accuracy (67.50% vs. 63.75% for Self-Refine and Budget Forcing). The multi-model variant further improves to 85.63% overall and 73.42% on Hard.

BabyVision. BabyVision evaluates image-conditioned reasoning over four task families and reports *Avg@3*, the official mean Pass@1 metric across three runs. Table 3 keeps the official overall-plus-family breakdown; Appendix, BabyVision gives the released split statistics, unified prompt template, Qwen3-Max judge protocol, and the paper's mean-plus-std reporting convention.

Table 3: BabyVision official references together with aligned reporting rows for the baselines and ATTS. Cited from Chen et al. [3] and the dataset card.

	Overall	Fine-gr.	Tracking	Spatial	Pattern	$/q
Human	94.1	92.3	94.6	94.7	97.8	–
Zero-Shot Models						
Gemini3-Pro-Preview	**49.7**	**46.2**	**43.4**	**53.7**	53.9	–
GPT-5.2	34.4	27.3	34.9	35.2	**54.9**	–
Doubao-1.8	30.2	39.2	15.7	24.7	37.7	–
Qwen3-VL-Plus	19.2	21.8	11.5	18.1	25.5	–
Grok-4	16.2	11.0	24.1	13.2	24.8	–
Test-Time Scaling Methods						
Pass@1	19.59	20.86	19.28	19.78	15.69	**0.07**
Majority Voting	22.42	**23.93**	21.69	23.08	**23.53**	0.50
Self-Refine	21.39	20.25	22.89	20.88	**23.53**	0.42
Budget Forcing	21.91	20.86	**28.92**	21.98	13.73	0.52
VisualPRM	20.88	20.86	21.69	21.98	17.65	0.50
ATTS (Ours)	**23.20**	22.09	24.10	**26.37**	19.61	0.27
ATTS Multi-model Med (Ours)	22.42	22.09	27.71	21.98	15.69	0.43

Figure 3: BabyVision cost-accuracy tradeoff.



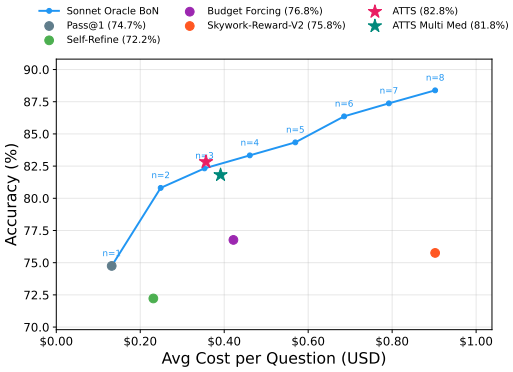
On BabyVision, ATTS achieves the highest overall accuracy among test-time scaling methods (23.20%) at $0.27/q, outperforming Majority Voting (22.42%) and Budget Forcing (21.91%). The largest per-subset gains appear on Spatial Perception, where ATTS reaches the highest accuracy (26.37%).

GPQA-Diamond. GPQA-Diamond is the 198-question highest-stringency subset of GPQA, evaluated as a closed-book four-option multiple-choice science benchmark with exact option-key Accuracy. Table 4 keeps the main-text comparison focused on this Diamond-set metric, while Appendix, GPQA-Diamond records the dataset release, subset construction, and the original paper's broader reporting context.

Table 4: GPQA-Diamond public references together with aligned reporting rows for the baselines and ATTS. Cited from Rein et al. [13] and official online records.

	Acc. (%)	$/q
Zero-Shot Models		
Gemini 3.1 Pro Preview	**94.1**	–
GPT-5.4 (xhigh)	92.0	–
GPT-5.3 Codex (xhigh)	91.5	–
Gemini 3 Pro Preview (high)	90.8	–
GPT-5.2 (xhigh)	90.3	–
Test-Time Scaling Methods		
Pass@1	74.24	**0.13**
Majority Voting	74.24	0.90
Self-Refine	72.22	0.23
Budget Forcing	76.77	0.42
Skywork-Reward-V2	75.76	0.90
ATTS (Ours)	**82.83**	0.35
ATTS Multi-model Med (Ours)	81.82	0.39

Figure 4: GPQA-Diamond cost-accuracy trade-off.



On GPQA-Diamond, ATTS achieves the highest accuracy among all test-time scaling methods (82.83%) at $0.35/q, substantially outperforming Budget Forcing (76.77%) and Skywork-Reward-V2 (75.76%). Notably, Self-Refine (72.22%) falls below Pass@1 (74.24%), consistent with the observation that its feedback model frequently accepts the initial draft without revision, and occasionally introduces errors when it does revise.

4.3 Overall Trends

Across benchmarks, ATTS is intended to occupy a middle ground between fixed-budget sequential refinement and purely parallel sampling. Direct one-shot generation is often already strong on easier exact-match tasks, but it can leave performance on the table once problems require either broader hypothesis coverage or evidence reconciliation across partially correct candidates. majority voting addresses the first issue by increasing candidate diversity, yet it still reduces the final decision to selection. The role of ATTS is to use the same test-time budget more adaptively: explore when the current pool is incomplete or inconsistent, and synthesize the final answer once the candidate set contains enough evidence.

This framing is especially useful on benchmarks where independently generated candidates can expose complementary reasoning fragments rather than merely redundant answers. On mathematical and scientific reasoning tasks, the expected gain comes from avoiding premature commitment to a single trajectory while still permitting a model-based final consolidation step. On code and multimodal reasoning tasks, the same controller provides a unified way to trade off breadth against synthesis without changing the benchmark-specific grading pipeline. Taken together, the results support the main claim of the paper: even a deliberately simple single-tool interface can be sufficient to turn test-time scaling into a semantic compute-allocation problem rather than a fixed sampling schedule.

4.4 Ablation Studies

Exploration effort. The multi-model variant exposes an *exploration effort* knob that controls how much extended thinking the orchestrator devotes to each allocation decision. Table 5 summarizes the effect across all four benchmarks at three effort levels (Low, Medium, High), holding the model pool and budget fixed.

Table 5: Effect of exploration effort on multi-model ATTS. Accuracy (%) and average cost per question ($/q) across benchmarks.

	Low		Medium		High	
	Acc.	$/q	Acc.	$/q	Acc.	$/q
HLE-Verified	44.00	1.52	57.00	1.63	**60.00**	2.38
LiveCodeBench	77.01	0.55	**85.63**	0.69	**85.63**	1.09
BabyVision	19.85	0.31	**22.42**	0.43	22.16	0.77
GPQA-Diamond	77.78	0.29	81.82	0.39	**82.32**	0.40

The Low-to-Medium transition yields the largest accuracy gain on every benchmark (up to +13 points on HLE, +8.6 on LiveCodeBench). Medium-to-High provides diminishing returns: on LiveCodeBench and BabyVision accuracy is flat while cost nearly doubles; only on HLE-Verified does High effort produce a meaningful further improvement (+3 points). Medium effort therefore represents the best cost-efficiency operating point across the benchmark suite. Figure 5 visualizes this pattern: the steep Low-to-Medium slope flattens into a plateau at Medium-to-High across all four benchmarks.

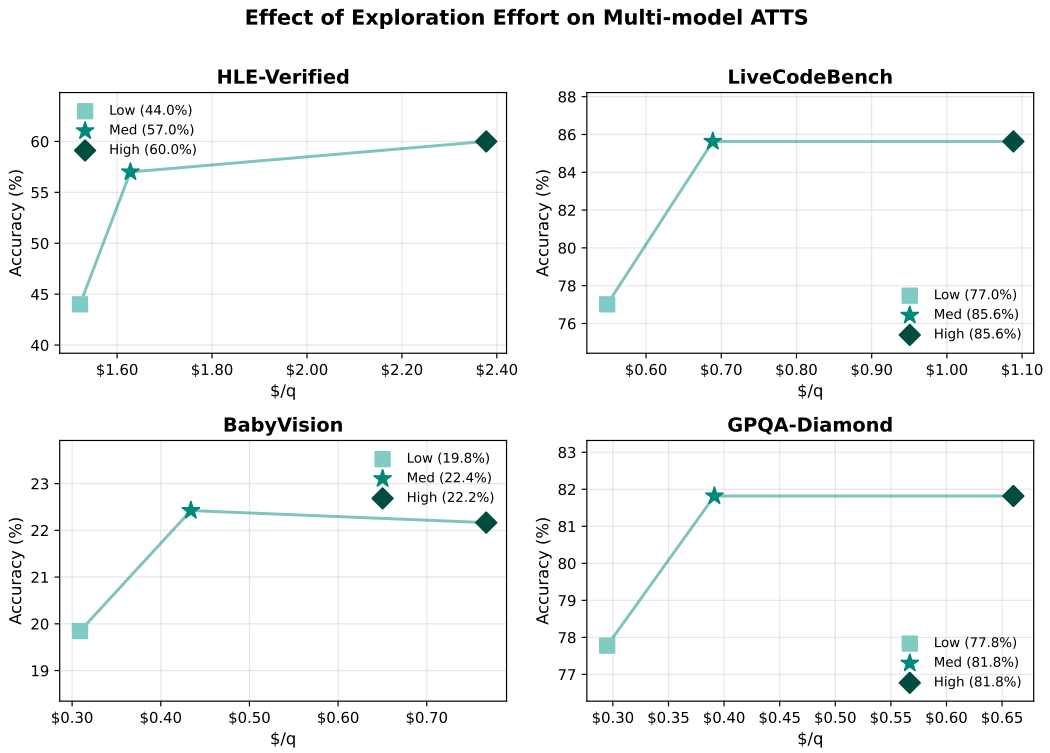


Figure 5: Effect of exploration effort on multi-model ATTS. Each panel shows the cost-accuracy tradeoff for Low, Medium, and High effort on one benchmark. The steep Low-to-Medium gain and flat Medium-to-High plateau are consistent across all benchmarks.

Adding a separate integrator. In the default ATTS architecture, the orchestrator produces the final answer directly from its accumulated evidence after deciding to stop exploring. We also evaluate a variant that adds a dedicated `integrate` step: a separate model call that receives all candidates and synthesizes the final answer independently. Across all four main-text benchmarks, adding a separate integrator does not consistently improve accuracy and always increases cost. On LiveCodeBench, accuracy drops from 81.71% to 80.57% while cost rises from $0.53/q to $0.72/q; on GPQA-Diamond, accuracy drops from 82.83% to 76.26% at $0.44/q (vs. $0.35/q). On HLE-Verified, the integrator

variant achieves 57.00% vs. 56.00% (+1 point) but at substantially higher cost ($2.66/q vs. $1.90/q); on BabyVision accuracy is identical (23.20%) with only cost increasing. The degradation likely stems from the integrator having less context for synthesis: it only sees the final candidate pool in a single-turn call, whereas the orchestrator has observed the full multi-turn conversation including each candidate's arrival and the reasoning behind its own exploration decisions.

Orchestrator model selection. We ablate the orchestrator model while keeping the explorer fixed at Sonnet 4.6. Table 6 reports accuracy and cost for Haiku 4.5, Sonnet 4.6, and Opus 4.6 as orchestrator on the three benchmarks where this ablation is available.

Table 6: Effect of orchestrator model on ATTS accuracy (%) and cost ($/q). Explorer is Sonnet 4.6 throughout.

	Haiku orch		Sonnet orch		Opus orch	
	Acc.	$/q	Acc.	$/q	Acc.	$/q
HLE-Verified	57.00	2.23	56.00	1.90	**59.00**	1.79
LiveCodeBench	79.43	0.71	81.71	0.53	**83.43**	0.64
GPQA-Diamond	79.80	0.36	**82.83**	0.35	77.27	0.34

Using Haiku 4.5 as orchestrator reduces per-call orchestrator cost but degrades accuracy: 79.8% vs. 82.8% on GPQA, 79.4% vs. 81.7% on LiveCodeBench. The accuracy loss is not solely due to weaker synthesis. Haiku dispatches significantly more explore calls on average (3.14 vs. 1.85 on LiveCodeBench), indicating it is less confident in its stopping decisions and compensates by requesting more evidence. The additional explore calls increase the total explorer cost, which more than offsets the savings from a cheaper orchestrator. This suggests that the orchestrator role demands strong meta-reasoning ability: a cheaper model saves on per-call cost but makes systematically worse allocation decisions. Figure 6 visualizes this tradeoff.

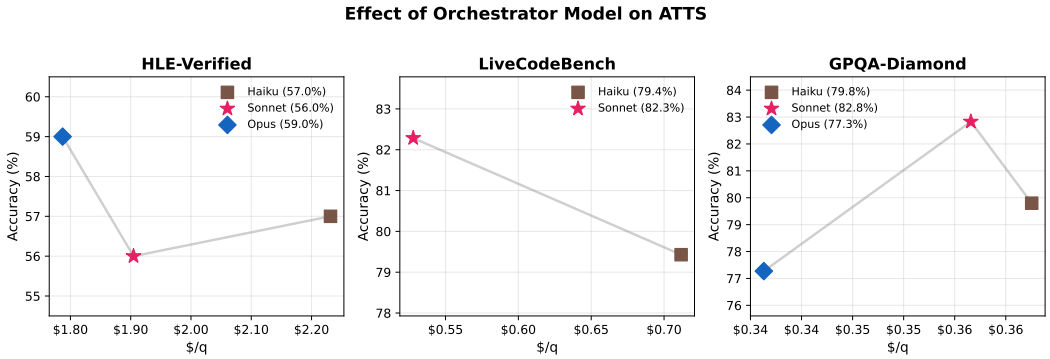


Figure 6: Effect of orchestrator model on ATTS accuracy and cost. Haiku is cheapest per call but over-explores, increasing total cost. Opus improves accuracy on HLE and LiveCodeBench but not on GPQA.

4.5 Analysis

Prompt design: principles over instructions. The orchestrator prompt uses a principles-based design rather than prescriptive instructions. Instead of specifying when to explore or stop (e.g. "explore at least twice before stopping"), the prompt states truths about the system (e.g. "a single candidate does not constitute sufficient evidence", "an answer without evidence from candidates is baseless") and provides worked examples showing how these principles apply in concrete scenarios. This design arose from empirical observation: prescriptive rules led to brittle behavior where the orchestrator would violate instructions in edge cases (such as attempting to solve problems itself when all solvers timed out), while principles-based prompts allowed the model to derive appropriate

behavior from its understanding of its role. The examples serve as demonstrations of principle application rather than exhaustive case coverage.

Multi-model exploration. The multi-model variant extends the ATTS framework to support heterogeneous model selection per explore call. In this variant, the orchestrator chooses which model (e.g. Haiku, Sonnet, or Opus) to dispatch for each exploration based on the current candidate pool state. The design follows the same principles-based approach: the orchestrator is informed that weaker models are cheaper but may fail on hard problems, and that escalation to stronger models is warranted when weaker models produce unsatisfactory results. This enables cost-aware exploration strategies where easy problems can be solved entirely with cheap models while hard problems automatically escalate to more capable solvers.

Conceptual framework: optimization in belief space. The ATTS orchestrator can be understood through the lens of *rational metareasoning* [14]: rather than solving the problem directly, it allocates computational resources (explore calls) to maximize expected answer quality under a budget constraint. Each explore result updates the orchestrator's *belief state*—its accumulated evidence about which answer is correct—and the next action is chosen based on the current belief. This sequential, memory-dependent structure distinguishes ATTS from memoryless strategies such as Best-of-N, where all candidates are generated independently and no intermediate evidence informs subsequent sampling.

From this perspective, the orchestrator performs *budget-constrained optimization in belief space*. Each explore functions as an "information gradient step" that reduces uncertainty about the correct answer, and the choice of action configuration (which model, at what cost) determines the step's magnitude and direction. Cheap models provide noisy but low-cost gradient signals; expensive models provide precise but costly ones. The stopping criterion—whether additional exploration has positive marginal value—is the natural analogue of a convergence check. This framing yields a concrete design principle: the value of the ATTS orchestrator is proportional to the richness of its action space. When the only action is "explore once more with the same model," the orchestrator can only decide *when* to stop. With heterogeneous models, it also decides *what* to compute. As the action space expands further (e.g. solver effort level, time budget, decomposition strategy), the orchestrator gains additional degrees of freedom in its optimization, enabling finer-grained adaptation to problem difficulty.

5 Limitations

ATTS inherits several limitations from the underlying language model and from its own controller design. First, the method can only synthesize evidence that is already present in the candidate pool, so if all explored candidates share the same misconception, the orchestrator has little opportunity to recover. Second, the current framework does not provide formal guarantees for stopping optimality, calibration, or budget efficiency; the controller is still a learned heuristic policy, even if its action space is intentionally simple. In practice, we observe that the orchestrator tends to converge early: on GPQA, 88% of questions use exactly two `explore` calls before stopping, even though the oracle best-of-N curve continues to climb through $N = 8$. This conservative exploration policy leaves headroom on the table, and learning a better stopping criterion remains an open problem. Third, because ATTS requires multiple model calls, it can increase latency and inference cost relative to direct generation on easy problems where extra exploration is unnecessary. Finally, the current evaluation focuses on benchmarked reasoning tasks with fixed answer interfaces rather than on open-ended tool-use agents, so the present results should be interpreted as evidence about reasoning-oriented test-time scaling rather than as a complete account of agentic decision making.

6 Conclusion

This paper introduced ATTS, a delegated sequential framework for test-time scaling in which a lightweight orchestrator adaptively controls exploration and directly synthesizes the final answer from an accumulated candidate pool. By separating control from solution generation and reducing the action space to a single `explore` tool, ATTS treats inference-time compute allocation as a stateful decision problem over a semantic pool of candidate hypotheses rather than as a fixed sampling

schedule or an explicit tree-search procedure. Across six reasoning benchmarks, this formulation provides a unified way to trade off candidate diversity against final evidence synthesis while preserving each benchmark's official evaluation protocol. More broadly, the results suggest that useful adaptive behavior can emerge from a minimal action space, without requiring hand-designed branching rules or dense search machinery.

References

[1] Center for AI Safety, Scale AI, and HLE Contributors Consortium. A benchmark of expert-level academic questions to assess AI capabilities. *Nature*, 649:1139–1146, 2026. doi: 10.1038/s41586-025-09962-4.

[2] Feng Chen, Allan Raventos, Nan Cheng, Surya Ganguli, and Shaul Druckmann. Rethinking fine-tuning when scaling test-time compute: Limiting confidence improves mathematical reasoning, 2025. URL https://arxiv.org/abs/2502.07154.

[3] Liang Chen, Weichu Xie, Yiyan Liang, et al. Babyvision: Visual reasoning beyond language, 2026. URL https://arxiv.org/abs/2601.06521.

[4] Yongchao Chen, Jiefeng Chen, Rui Meng, Ji Yin, Na Li, Chuchu Fan, Chi Wang, Tomas Pfister, and Jinsung Yoon. Tumix: Multi-agent test-time scaling with tool-use mixture, 2025. URL https://arxiv.org/abs/2510.01279.

[5] Audrey Huang, Adam Block, Qinghua Liu, Nan Jiang, Dylan J. Foster, and Akshay Krishnamurthy. Is best-of-n the best of them? coverage, scaling, and optimality in inference-time alignment, 2025. URL https://arxiv.org/abs/2503.21878.

[6] Naman Jain, Kevin Han, Alex Gu, et al. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *ICLR*, 2025. URL https://arxiv.org/abs/2403.07974.

[7] Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Agentic test-time scaling for webagents, 2026. URL https://arxiv.org/abs/2602.12276.

[8] Xiaochuan Li, Ryan Ming, Pranav Setlur, Abhijay Paladugu, Andy Tang, Hao Kang, Shuai Shao, Rong Jin, and Chenyan Xiong. Benchmark test-time scaling of general LLM agents, 2026. URL https://arxiv.org/abs/2602.18998.

[9] Chris Yuhao Liu, Liang Zeng, Yuzhen Xiao, Jujie He, Jiacai Liu, Chaojie Wang, Rui Yan, Wei Shen, Fuxiang Zhang, Jiacheng Xu, Yang Liu, and Yahui Zhou. Skywork-reward-v2: Scaling preference data curation via human-ai synergy, 2025. URL https://arxiv.org/abs/2507.01352.

[10] Xiang Liu, Xuming Hu, Xiaowen Chu, and Eunsol Choi. Diffadapt: Difficulty-adaptive reasoning for token-efficient LLM inference, 2025. URL https://arxiv.org/abs/2510.19669.

[11] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf.

[12] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20275–20321. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.1025. URL https://aclanthology.org/2025.emnlp-main.1025/.

[13] David Rein, Betty Li Hou, Asa Cooper Stickland, et al. Gpqa: A graduate-level google-proof q&a benchmark. In *ICLR*, 2024. URL https://arxiv.org/abs/2311.12022.

[14] Stuart J. Russell and Eric Wefald. Do the right thing: Studies in limited rationality. Cambridge, MA, 1991. MIT Press.

[15] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2408.03314.

[16] Richard Sutton. The bitter lesson. http://www.incompleteideas.net/IncIdeas/BitterLesson.html, 2019.

[17] Guiyao Tie, Zenghui Yuan, Zeli Zhao, Chaoran Hu, Tianhe Gu, Ruihang Zhang, Sizhe Zhang, Junran Wu, Xiaoyue Tu, Ming Jin, Qingsong Wen, Lixing Chen, Pan Zhou, and Lichao Sun. Can LLMs correct themselves? a benchmark of self-correction in LLMs, 2025. URL https://arxiv.org/abs/2510.16062.

[18] Siddarth Venkatraman, Vineet Jain, Sarthak Mittal, Vedant Shah, Johan Obando-Ceron, Yoshua Bengio, Brian R. Bartoldson, Bhavya Kailkhura, Guillaume Lajoie, Glen Berseth, Nikolay Malkin, and Moksh Jain. Recursive self-aggregation unlocks deep thinking in large language models, 2025. URL https://arxiv.org/abs/2509.26626.

[19] Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Yiran Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations, 2023. URL https://arxiv.org/abs/2312.08935.

[20] Weiyun Wang, Zhangwei Gao, Lianjie Chen, Zhe Chen, Jinguo Zhu, Xiangyu Zhao, Yangzhou Liu, Yue Cao, Shenglong Ye, Xizhou Zhu, Lewei Lu, Haodong Duan, Yu Qiao, Jifeng Dai, and Wenhai Wang. Visualprm: An effective process reward model for multimodal reasoning, 2025. URL https://arxiv.org/abs/2503.10291.

[21] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2203.11171.

[22] Yuxi Xie, Anirudh Goyal, Wenyue Zheng, Min-Yen Kan, Timothy P. Lillicrap, Kenji Kawaguchi, and Michael Shieh. Monte carlo tree search boosts reasoning via iterative preference learning, 2024. URL https://arxiv.org/abs/2405.00451.

[23] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023. URL https://arxiv.org/abs/2305.10601.

[24] Weiqi Zhai, Zhihai Wang, Jinghang Wang, et al. Hle-verified: A systematic verification and structured revision of humanity's last exam, 2026. URL https://arxiv.org/abs/2602.13964.

A Implementation Details

Controller and backend setup. ATTS is implemented as a delegated controller with two roles: an orchestrator and an explorer. The orchestrator never solves the task directly; it only decides whether to spend the remaining budget on another `explore` call or to stop and synthesize the final answer from the accumulated evidence. Each `explore` call returns a structured candidate containing an answer, supporting reasoning, an approach summary, and an optional confidence score. When the orchestrator decides to stop, it produces the final answer directly via the SDK's structured output mechanism. For the Claude implementation, we use the Claude Agent SDK[5] and expose `explore` as the only available delegated tool. We also evaluate a variant that adds a separate `integrate` tool that delegates final synthesis to a dedicated model call; see the ablation in Section 4.4. The same high-level controller is reused across benchmarks, while the underlying sub-model backend can be swapped without changing the controller logic.

[5] https://docs.anthropic.com/en/docs/claude-code/sdk

Prompt families. ATTS uses two prompt families: a benchmark-agnostic orchestrator prompt and a benchmark-specific explorer prompt. The orchestrator prompt defines the controller role and explicitly instructs it not to solve the task itself. The explorer prompt asks for an independent solution from scratch under the target answer format. Representative prompt excerpts are shown in Listings 1 and 2. For Claude runs, these prompts are paired with structured-output schemas so that each sub-call returns machine-readable fields rather than free-form text only.

```
You are a meta-reasoning orchestrator.
You manage a pool of candidate solutions for a problem.

You have one tool:
1. explore -- Dispatch a fresh, independent solver.

Keep calling explore until you have enough evidence,
then submit your final answer directly.
NEVER attempt to solve the problem yourself.
```

Listing 1: Shared orchestrator prompt excerpt.

```
Explorer:
You are an expert problem solver. Solve the given problem step by step
    .
Return the final answer in a dedicated short-answer field.
```

Listing 2: Explorer prompt pattern.

Schemas and benchmark-specific prompt instantiation. Benchmark-specific adaptation is intentionally lightweight. Most answer-based tasks share the explorer schema `{approach, reasoning, answer, confidence}` and the integrator schema `{analysis, final_answer, reasoning}`. Only the answer interface changes across tasks. LiveCodeBench uses code-generation fields, AIME requires an integer final answer, GPQA expects one option letter, and multimodal benchmarks pass the associated image with the text question. This design keeps the controller fixed while preserving each benchmark's native answer format and grading protocol.

Runtime behavior. At runtime, the controller maintains the candidate pool and repeatedly calls `explore` until the exploration budget is exhausted or the controller decides to stop and synthesize. The same implementation supports both text-only and image-conditioned tasks, and all runs can optionally record the controller trace and sub-model outputs for later inspection. These implementation choices are meant to make the evaluation pipeline inspectable without changing the central adaptive exploration decision process.

Baseline implementations. All baselines use the same backbone model and the same benchmark-specific answer format as ATTS, so the comparison isolates the inference-time strategy.

Pass@1 generates a single independent response per question and returns its answer directly.

Majority Voting draws $N = 8$ independent responses and selects the most frequent answer.

Self-Refine [11] generates an initial draft, then enters a feedback-revision loop: a separate feedback call critiques the current draft and returns a correctness judgment; if the draft is judged incorrect, a revision call produces an updated draft conditioned on all prior drafts and feedbacks. The loop terminates when feedback returns a positive judgment or the iteration budget is exhausted. In practice, the feedback model frequently accepts the initial draft, so Self-Refine terminates after a single feedback call on the majority of questions, which explains its relatively low cost on most benchmarks.

Budget Forcing [12] runs N sequential rounds on the same question. Round 1 generates an initial solution. Each subsequent round feeds the previous round's full reasoning trace back to the model with "Wait" appended, suppressing premature termination and forcing the model to continue deliberating before producing a new answer. All N rounds run unconditionally, and the final round's answer is returned. Because continuation rounds extend the existing reasoning rather than solving from scratch, they tend to produce shorter completions and lower per-round cost.

Reward-model reranking (Skywork-Reward-V2 for text benchmarks, VisualPRM for multimodal benchmarks) draws the same $N = 8$ independent candidates, then scores each with a separately trained reward model and returns the highest-scoring answer. Skywork-Reward-V2 is an outcome reward model that assigns a single scalar score per response; VisualPRM is a process reward model that aggregates per-step scores. The reward models run locally and incur no API cost, but the candidate generation cost is identical to that of Majority Voting.

B AIME Results

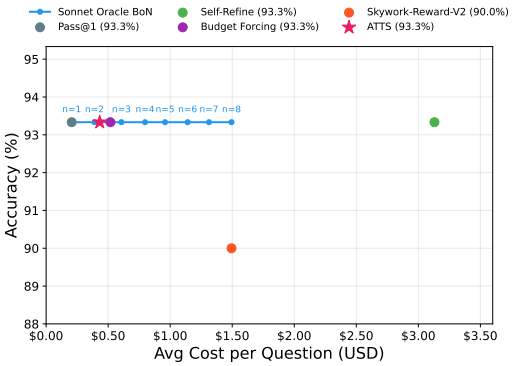
AIME 2025 and AIME 2026 are 30-problem final-answer mathematics benchmarks with exact integer-answer grading. Because the backbone model already achieves near-ceiling accuracy on these benchmarks (93.33% on AIME 2025 with a single attempt), the statistical resolution for method comparison is limited. We include them here for completeness.

AIME 2025. Table 7 reports the public Accuracy metric; Appendix, AIME 2025 details records the official answer-space constraint, source repositories, and leaderboard alignment.

Table 7: AIME 2025 public references together with aligned reporting rows for the baselines and ATTS. Cited from official online records.

	Acc. (%)	$/q
Zero-Shot Models		
GPT-5.2 (high)	**100.00**	–
Step 3.5 Flash	98.33	–
Gemini 3 Flash	97.50	–
GLM 5	96.67	–
DeepSeek-v3.2-Speciale	95.83	–
Test-Time Scaling Methods		
Pass@1	**93.33**	**0.21**
Majority Voting	**93.33**	1.49
Self-Refine	**93.33**	3.13
Budget Forcing	**93.33**	0.52
Skywork-Reward-V2	90.00	1.49
ATTS (Ours)	**93.33**	0.43

Figure 7: Cost-accuracy tradeoff on AIME 2025.



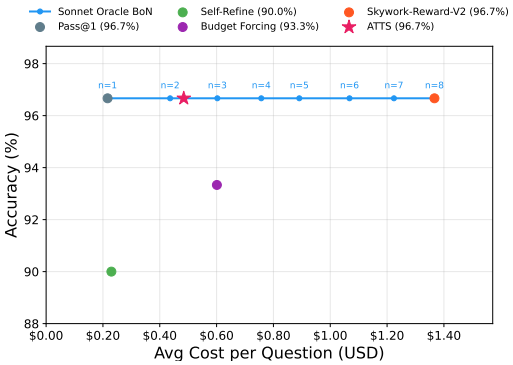
On AIME 2025, most test-time scaling methods converge to the same accuracy (93.33%), with only Skywork-Reward-V2 falling behind at 90.00%. The 30-problem dataset leaves little statistical room for differentiation, and the backbone model already solves 28 of 30 problems with a single attempt.

AIME 2026. AIME 2026 uses the same official final-answer setup as AIME 2025: 30 released problems, one integer answer per problem, and exact-key Accuracy. Table 8 reports the corresponding public leaderboard references; Appendix, AIME 2026 details provides the full source and grading details.

Table 8: AIME 2026 public references together with aligned reporting rows for the baselines and ATTS. Cited from official online records.

	Acc. (%)	$/q
Zero-Shot Models		
GPT-5.4 (xhigh)	**99.17**	–
GPT-5.2 (high)	98.33	–
Gemini 3.1 Pro Preview	98.33	–
Step 3.5 Flash	96.67	–
Claude-Opus-4.6 (High)	96.67	–
Test-Time Scaling Methods		
Pass@1	93.33	**0.22**
Majority Voting	93.33	1.37
Self-Refine	90.00	0.23
Budget Forcing	93.33	0.60
Skywork-Reward-V2	**96.67**	1.37
ATTS (Ours)	**96.67**	0.48

Figure 8: Cost-accuracy tradeoff on AIME 2026.



On AIME 2026, ATTS and Skywork-Reward-V2 both reach 96.67%, improving over Pass@1 (93.33%). Self-Refine drops to 90.00%, likely because its feedback model misjudges some correct solutions and introduces errors during revision. As with AIME 2025, the small dataset size (30 problems) limits the granularity of method comparisons.

C Benchmark Details

HLE-Verified. We use the official HLE-Verified dataset release `skylenage/HLE-Verified`, which partitions the 2,500 audited HLE items into Gold (668), Revision (1,143), and Uncertain (689) subsets [24]. HLE-Verified preserves the original HLE answer-based evaluation semantics: each item contains a problem statement, a final answer, and sometimes a rationale, but official scoring is driven by the problem and final answer, with rationale used only as auxiliary evidence during verification and revision [1, 24]. In the HLE-Verified evaluation paper, the reported baselines are run on the text-only portion of HLE using the official HLE system prompt, each model's default recommended decoding configuration, and five independent rollouts per item; the paper reports both *avg5 Accuracy* and *Calibration Error*, with calibration computed by the official HLE evaluation code [24]. In this paper, HLE reporting is aligned to the Gold-subset setting, while Table 1 reproduces the official revised-subset accuracy references from Zhai et al. [24] for context. Our grading uses an LLM-as-judge pipeline with Claude Haiku 4.5 as the judge model, which determines semantic equivalence between the predicted and gold answers.

LiveCodeBench. We use the LiveCodeBench code generation release `livecodebench/code_generation_lite`, configuration `v6` [6]. LiveCodeBench is a continuously updated programming benchmark sourced from LeetCode, AtCoder, and CodeForces. Each problem is tagged by release date, so official evaluations can be restricted to post-cutoff time windows for contamination-aware comparison [6]. In the official code generation setting, the model receives the natural-language problem statement together with example input-output pairs and must return a complete executable program. Correctness is determined purely by functional correctness on unseen tests, using the benchmark harness rather than an LLM judge [6]. The benchmark paper reports *Pass@1* as the primary metric and also reports difficulty-stratified *Pass@1* using the platform-provided Easy/Medium/Hard labels. The public generation leaderboard follows this same framing, and Table 2 therefore reports overall and difficulty-wise Pass@1 from the same leaderboard view. Our grading uses the official `lcb_runner` execution harness with a 10-second per-test-case timeout.

BabyVision. We use the BabyVision dataset release `UnipatAI/BabyVision`, which contains 388 image-question pairs spanning four task families: Fine-grained Discrimination (163), Visual Tracking (83), Spatial Perception (91), and Visual Pattern Recognition (51) [3]. The benchmark contains 135 multiple-choice items and 253 fill-in-the-blank items; in the official inference setup, each model is queried with a unified prompt template of the form "`{question} Think about the question and give your final answer in {format}.`", which standardizes answer formatting for downstream extraction and judging [3]. Official scoring uses an LLM-as-judge pipeline: given a model answer and the gold answer, Qwen3-Max determines semantic equivalence under the benchmark's fixed judge prompt, which the authors report as having perfect agreement with their human evaluators on the benchmark responses they audited [3]. The primary metric is *Avg@3*, defined in the paper as the mean *Pass@1* accuracy across three random runs, and the official tables additionally report the corresponding standard deviation; Table 3 follows the official overall-plus-family breakdown but keeps only the mean scores in the main text table. Our grading uses an LLM-as-judge pipeline with Claude Haiku 4.5 as the judge model, following the same semantic equivalence protocol as the official evaluation but substituting the judge model.

AIME 2025. AIME 2025 is treated as a final-answer mathematics benchmark on the MathArena competition table, which lists 30 problems for the 2025 competition and notes that each answer is an integer between 0 and 999 inclusive[6]. The task is constrained final-answer prediction: the model receives the official problem statement and is expected to return one final integer answer in the AIME

[6] `https://matharena.ai/competition_tables/aime--aime_2025`
`https://huggingface.co/datasets/yentinglin/aime_2025`

answer space. Official grading is exact-answer grading against the released answer key, with no LLM judge and no partial credit, so each problem contributes a binary correct/incorrect label. The public leaderboard metric is *Accuracy*, namely the percentage of the 30 official problems answered exactly correctly; Table 7 reports that same public leaderboard metric. Our grading uses exact integer matching with no LLM judge, consistent with the official protocol.

AIME 2026. AIME 2026 appears on MathArena as another 30-problem final-answer competition, again with the official note that each answer is an integer between 0 and 999 inclusive[7]. The official task semantics match AIME 2025: the model is given the problem statement and must return one final integer answer. The benchmark performs exact-key grading with no LLM judge and no partial credit, and the leaderboard metric is therefore plain *Accuracy* over the 30 released competition problems. Keeping the same final-answer protocol across 2025 and 2026 allows direct year-to-year comparison under the benchmark's official grading rule; the corresponding public references are shown in Table 8. Our grading uses exact integer matching, consistent with the official protocol.

GPQA-Diamond. We use the GPQA release `Idavidrein/gpqa`, configuration `gpqa_diamond`, which corresponds to the 198-question Diamond subset introduced in the GPQA paper [13]. GPQA is a graduate-level four-option multiple-choice science benchmark spanning biology, physics, and chemistry, and GPQA-Diamond is the highest-stringency subset retained after expert-accuracy and non-expert-difficulty filtering [13]. The official task is closed-book answer selection unless otherwise stated: the model is shown the question and four candidate options and must output exactly one final option. Scoring is exact option-key accuracy with no semantic judge and a random-guess baseline of 25% [13]. In addition to Diamond-set accuracy, the original GPQA paper also reports accuracies on the Extended and Main sets and distinguishes closed-book few-shot/CoT baselines from an open-book GPT-4-with-search baseline; Table 4 keeps the main-text comparison focused on the Diamond-set accuracy column used by current public references. Our grading extracts the option letter (A–D) from the model response via regex and performs exact matching against the gold option key, with no LLM judge, consistent with the official protocol.

[7] `https://matharena.ai/competition_tables/aime--aime_2026`
`https://huggingface.co/datasets/MathArena/aime_2026`

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [NA]

   Justification: To be completed in the final submission.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [NA]

   Justification: To be completed in the final submission.

3. **Theory Assumptions and Proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [NA]

   Justification: To be completed in the final submission.

4. **Experimental Result Reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [NA]

   Justification: To be completed in the final submission.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [NA]

   Justification: To be completed in the final submission.

6. **Experimental Setting/Details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [NA]

   Justification: To be completed in the final submission.

7. **Experiment Statistical Significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [NA]

   Justification: To be completed in the final submission.

8. **Experiments Compute Resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [NA]

   Justification: To be completed in the final submission.

9. **Code Of Ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [NA]

Justification: To be completed in the final submission.

10. **Broader Impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [NA]

    Justification: To be completed in the final submission.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: [NA]

    Justification: To be completed in the final submission.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [NA]

    Justification: To be completed in the final submission.

13. **New Assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [NA]

    Justification: To be completed in the final submission.

14. **Crowdsourcing and Research with Human Subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: To be completed in the final submission.

15. **Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

    Justification: To be completed in the final submission.